

Python - Floating Points

When working with the numbers, we will come across the **floating-point numbers**. These are the numbers that have a decimal point like 1.12, 12.01 or -321.243. Floating-point are widely used in the daily calculations as they allows us to represent both very large and very small values in the convenient way.

However, the floating-point arithmetic can sometimes behaves in a ways that cause surprise while observing the outcome. For example, we expect simple operations like **0.1+0.2= 0.3**, but the python tells it's 0.30000000000000004. This isn't a bug in Python it is a result of how computers represent decimal numbers internally.

Floating-Point Arithmetic

Floating-point arithmetic refers to the calculations involving numbers with fractional parts, typically represented in a format based on the **IEEE 754 standard**. A floating-point number is stored in memory using a fixed number of binary digits (bits).

In Python, the default floating-point type is called float and uses 64 bits. The number is broken into three parts:

- **Sign bit** – It indicates whether the number is positive or negative.
- **Exponent** – It indicates the scale of the number up or down.
- **Mantissa** – It stores the digits of the number.

This format allows the python to represent an wide range of values (from about 10^{-308} to 10^{308}) and handle very small increments between them.

Issues and Limitations

Let's discuss some of the main issues of the floating-point arithmetic:

Precision Errors

Since the float uses the finite number of bits, they can't store every decimal number exactly. which leads to the tiny rounding errors.

```
print(0.1 + 0.2)    # Expected output: 0.3
```

The **output** of the above program is -

```
0.30000000000000004
```

In this case, the result is slightly off because 0.1 and 0.2 don't have exact binary representations.

Comparisons Issues

Because of the precision issues, the direct comparisons of the floating-point numbers often fail.

```
a = 0.1 + 0.2
print(a == 0.3)
```

The **output** of the above program is -

```
False
```

Loss of Significance

In this scenario, when we subtract two nearly equal floating-point numbers, small differences can be magnified, leading to loss of significance.

```
a = 2.000001
b = 2.000000
print(a - b)
```

The **output** of the above program is -

```
1.000000000139778e-06
```

Overflow and Underflow

The floating-point numbers have maximum and minimum representable values. If the calculation exceeds these limits:

- **Overflow** – This number is too large, results in infinity.
- **Underflow** – The number is too close to the zero results in 0.0

```
large = 1e3211
print(large * 10) # Overflow
small = 1e-213
print(small / 10) # Underflow
```

The **output** of the above program is -

```
inf
1e-214
```

Examples of Using Floating-Point Arithmetic

Let's explore some of the examples to understand more about the floating-point arithmetic.

Example 1

Consider the following example, where we are going to use the `round()` function.

```
a = 0.1 + 0.2
print(a)
```

```
print(round(a, 2))
```

The **output** of the above program is -

```
0.30000000000000004  
0.3
```

Example 2

In the following example, we are going to use the `math.isclose()` method to check whether the two numbers are close enough or not.



```
import math  
a = 0.1 + 0.2  
print(math.isclose(a, 0.3))
```

Following is the **output** of the above program -

```
True
```

TOP TUTORIALS

[Python Tutorial](#)

[Java Tutorial](#)

[C++ Tutorial](#)

[C Programming Tutorial](#)

[C# Tutorial](#)

[PHP Tutorial](#)

[R Tutorial](#)

[HTML Tutorial](#)

[CSS Tutorial](#)

[JavaScript Tutorial](#)

[SQL Tutorial](#)

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps
Advanced Certification In Business Analytics



Chapters ▾

Categories ≡

Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler

[Online Bash Terminal](#)

[Online SQL Compiler](#)

[Online Html Editor](#)

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.